

A Prediction Based Replica Selection Strategy for Reducing Tail Latency in Geo-Distributed Systems

Santa Maria Shithil, Muhammad Abdullah Adnan

Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology

Content

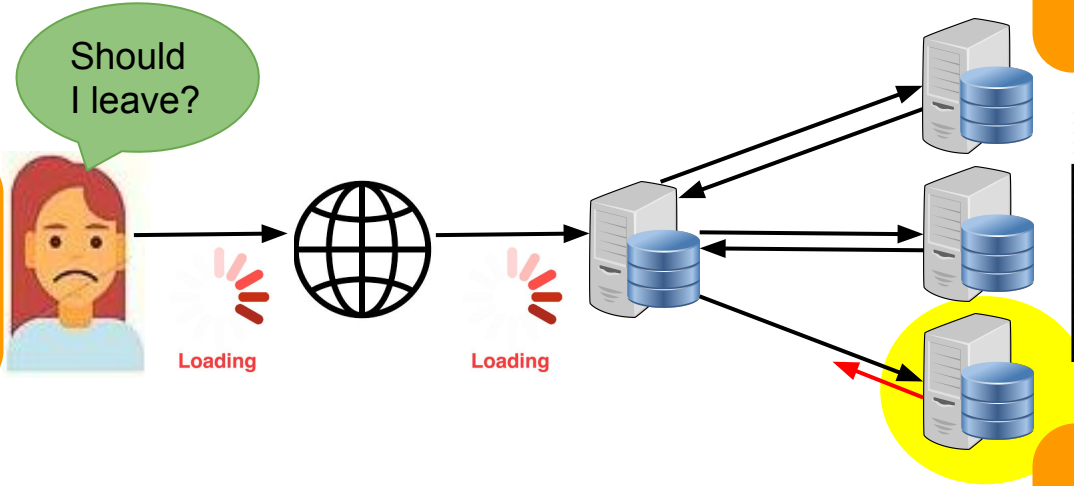
- Introduction and Problem Statement
- Objective
- Background
- Related Work
- Methodology
- Experiment Design
- Significant Results
- Conclusion

Introduction & Problem Statement

Server Performance Fluctuations are a Norm in Geo-Distributed Systems

at Amazon, every 100 milliseconds of latency causes a 1% decrease in sale [2]

over 40% of web users abandon a site if it takes more than 3 seconds to load[1]



- Affects applications
- Impacts the user experience
- Reduce revenue of the service provider

at Bing, a two-second slowdown was found to reduce revenue per user by 4.3% [2]

"Tail Latency" is the devil here

Overall Latency = 3ms

The Larger a System Grew in Scale, the Wider the Degree of Latency Variability [3]

1.3 μ s

1 μ s

2.1 μ s

1.3 μ s

1 μ s

3ms

1.6 μ s

Of Data Accesses

Objective

Is it possible to reduce tail latency?

YES

Ways to Reduce Tail Latency

Request Reissue

If multiple downstream requests hit a single service affected with long tail latencies, then the problem becomes scarier

Request Duplication

Selecting poorly performing nodes makes the problem more severe

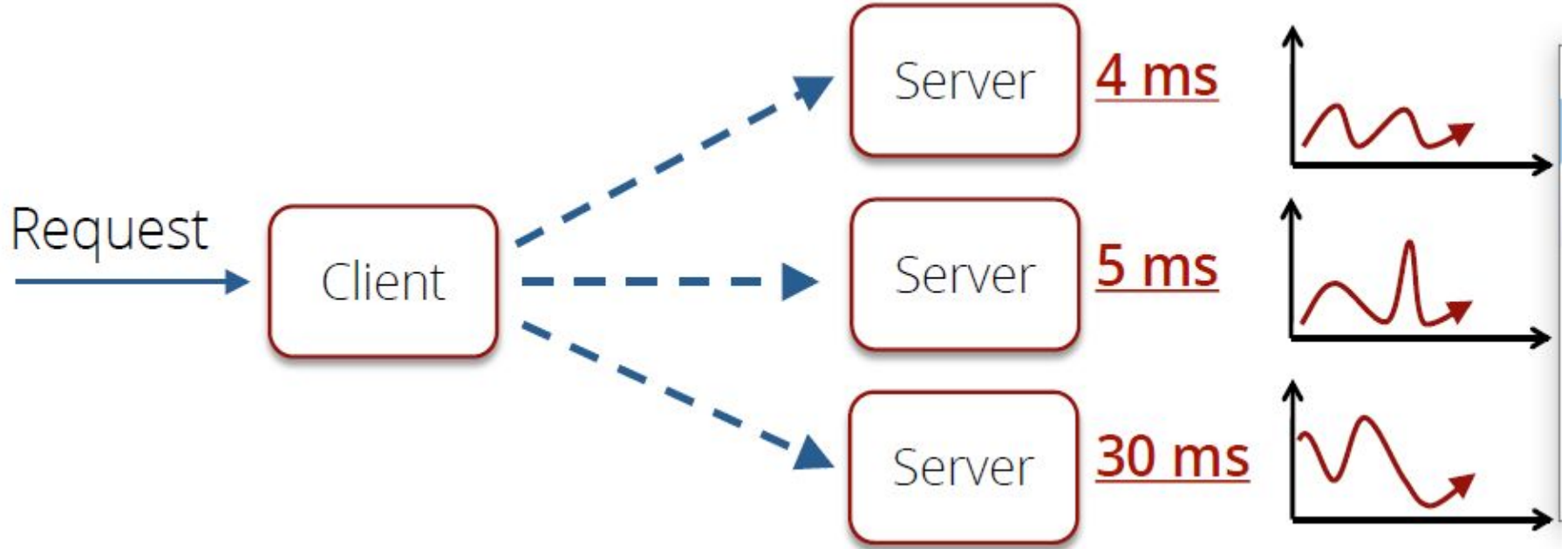
Replica Selection

Can select best replica. Is a first order concern in this context.

Background

Challenges of Replica Selection

Service-time variations and Load oscillations



Factors to Consider During Replica Selection

- Round trip time (RTT)
- Service time
- Queue size
- Disk I/O operations rate(IOPS)

Related Work

Dynamic Snitch [4]

- Only considered latency as performance metric
- Take the median of the latency
- Still there is latency spike
- System can not respond to time varying performance fluctuations

C3 [5]

- The score is monopolistically dependent on queue size
- Calculated queue size drifted away from the original queue size
- Calculated score does not reflect the actual status of the replicas
- Totally dependent on past information

Efficient Replica Selection Strategy

- Considers only number of replicas and response time
- Requires global information
- Can not response to time-varying performance fluctuations

- Calculated queue size drifted away from the original queue size
- Calculated score does not reflect the actual status of the replicas
- Totally dependent on past information

A Prediction Based Replica Selection Strategy for Reducing Tail Latency in Geo-Distributed Systems

Methodology

Considered Performance Metrics

- Round trip time (RTT)
- Service time
- Queue size
- Disk I/O operations

A Novel Prediction Based Strategy

- Predicts the latency of individual server
- Round trip time, Service time and queue size are used as dependent variable
- Calculates a score for each server based on the predicted latency and IOPS
- Service time, queue size and IOPS of each server are piggybacked to the client
- Score is calculated and used by the clients for selecting the best replica

Predicting Latency

$$L = RTT_s + R_s$$

$$L_{ps} = a + b \cdot RTT_s + c \cdot R_s$$

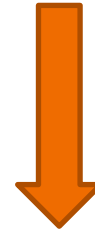
$$a = \bar{L}_s - b \cdot \bar{RTT}_s - c \cdot \bar{R}_s$$

$$b = \frac{\sum R_s^2 \cdot \sum (RTT_s \cdot L_s) - \sum (RTT_s \cdot R_s) \cdot \sum (L_s \cdot R_s)}{\sum RTT_s^2 \cdot \sum R_s^2 - (\sum (RTT_s \cdot R_s))^2}$$

$$c = \frac{\sum RTT_s^2 \cdot \sum (L_s \cdot R_s) - \sum (RTT_s \cdot R_s) \cdot \sum (RTT_s \cdot L_s)}{\sum RTT_s^2 \cdot \sum R_s^2 - (\sum (RTT_s \cdot R_s))^2}$$

Smoothing the Predicting Latency

$$L_{ps} = a + b \cdot RTT_s + c \cdot R_s$$



$$\hat{L}_{ps} = a + b \cdot \hat{RTT}_s + c \cdot \hat{R}_s$$

To make the prediction more accurate we use Exponentially Weighted Moving Average (EWMA) of both response time and RTT instead of using these directly

Calculating the Final Score

$$\phi_s = \hat{L}_{ps} + \hat{R}_{ios}$$

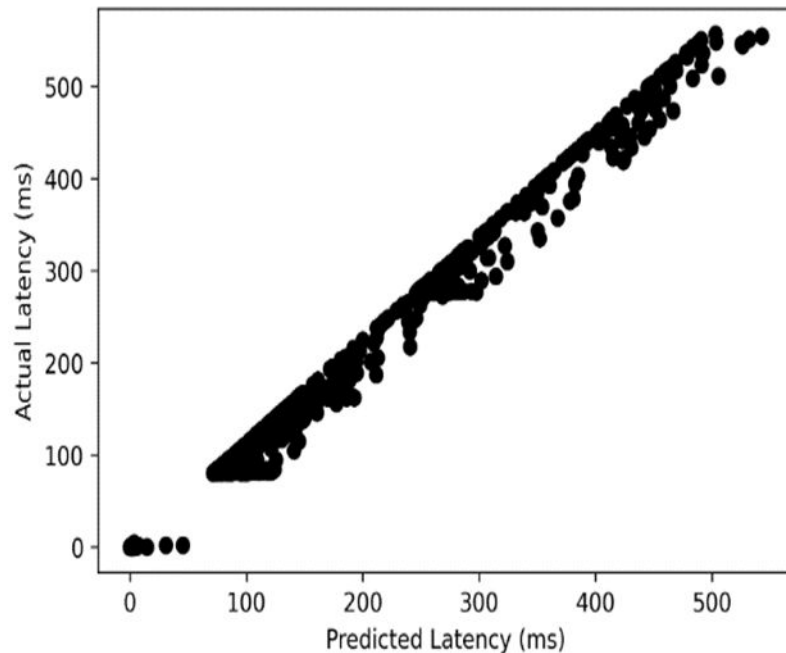
Final score

EWMA of
predicted latency

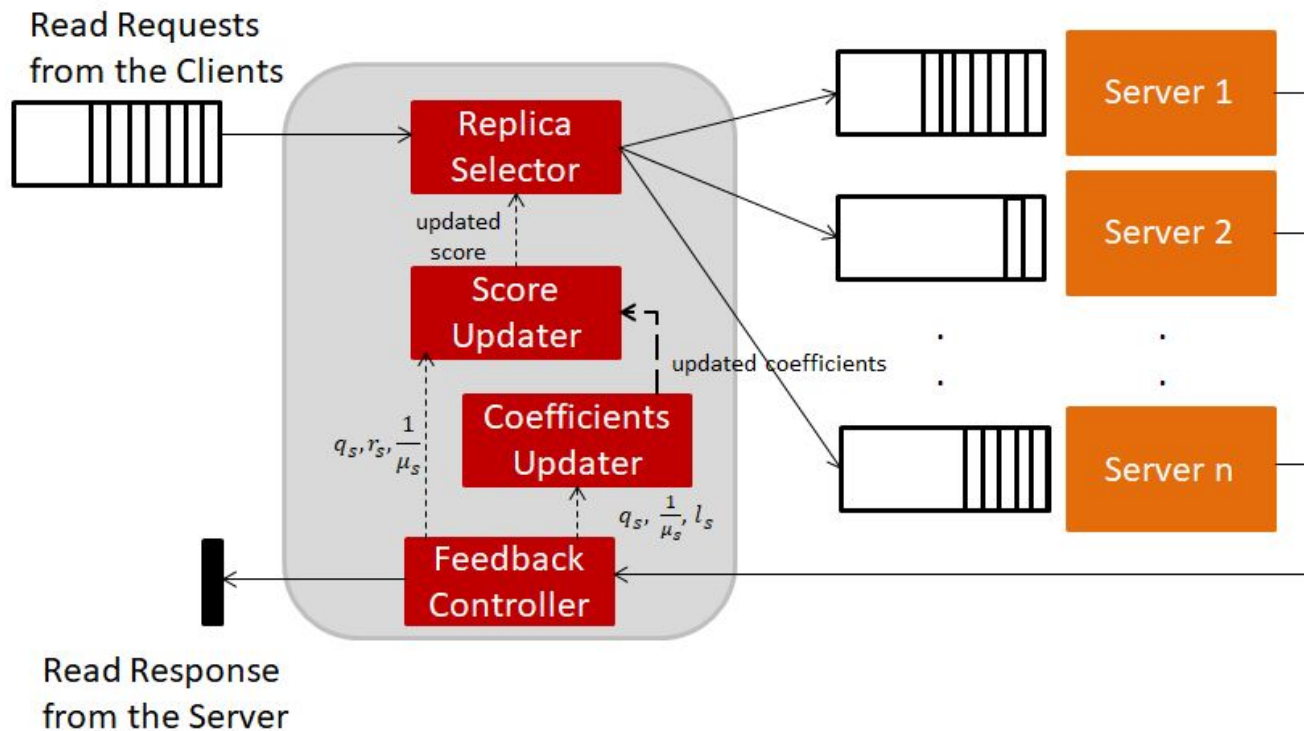
EWMA of disk I/O
per second

Nodes with the lowest
score will be selected
for routing the read
request

Predicted latency vs actual latency



Putting Everything Together



Experiment Design

Experimental Setup

Cluster Configuration

- Deployed a Cassandra cluster in Amazon EC2 with 15 c5d.large instances
- Number of racks: 5
- Racks were geographically distributed across Singapore, US East (N. Virginia), Europe (Paris), Canada(Central), and Australia (Sydney).
- Operating system: Ubuntu 16.04
- Snitch: GossipingPropertyFileSnitch
- Replication factor: 3
- Number of token: 256

Implementation

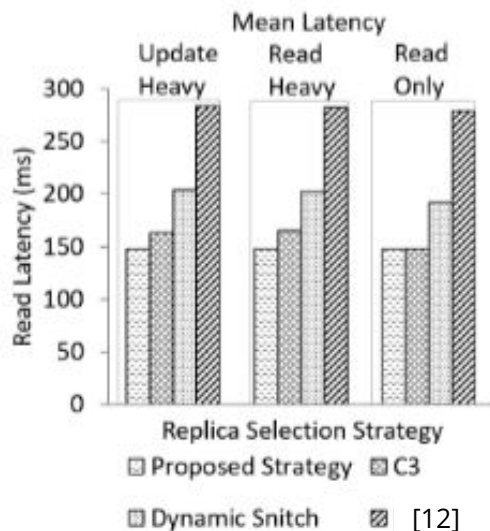
- Chosen distributed database system: Cassandra 3.0 [9]
- We modified the source code of Cassandra in many places and implemented several new methods [10]

Experimental Setup: Benchmarking

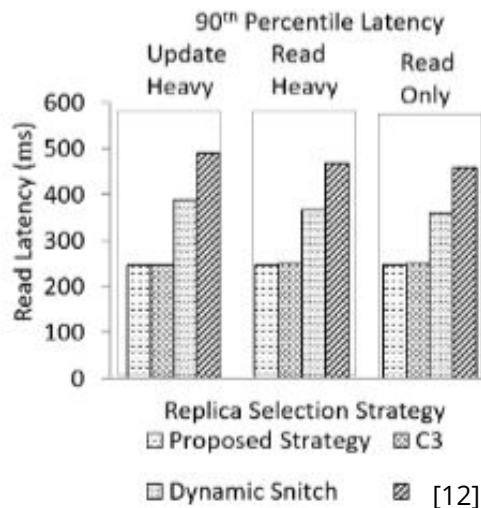
- Yahoo Cloud Serving Benchmark (YCSB) [11]
- Used workloads:
 - Workload A (update-heavy 50% reads and 50% writes)
 - Workload B (read-heavy 95% reads and 5% writes)
 - Workload C (read-only 100% reads)
- Loaded Test Data:
 - Total record count = 20000000 (20 million)
 - Field length = 1KB
- Executed workloads:
 - 30 request generator
 - Total operation count = 10000000 (10 million)
- Used distribution: Zipfian with Zipf parameter
- Each experiment repeated for 5 times

Significant Results

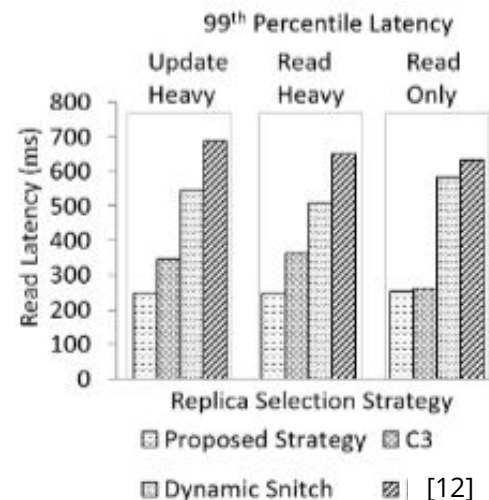
Comparison of Latency at Different Workloads



(a)



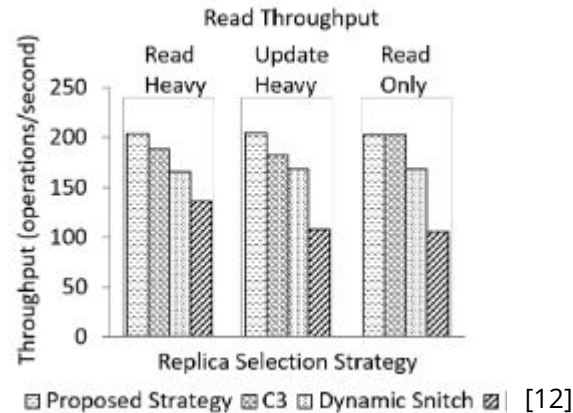
(b)



(c)

Latency of Cassandra when using C3, dynamic snitch, [12] and our proposed strategy
(a) mean latency (b) 90th percentile latency (c) 99th percentile latency.

Comparison of Throughput at Different Workloads



Comparison of read throughput at different work-loads. Our proposed strategy has higher throughput at all the scenarios

Conclusion

Contributions

- We developed a novel prediction based replica selection strategy
- First strategy that integrates almost all the important performance metrics (i.e. latency, queuesize, service time, disk I/O operations)
- Calculates score that reflects the actual status of the replicas and reduces latency
- Improves the ability to react more efficiently to the time varying performance fluctuations

Next Works

- A study is required to determine the standard time interval of coefficients recalculation and update

References

- [1] <https://phoenixnap.com/kb/reduce-server-response-time>
- [2] <https://www.computerweekly.com/opinion/Storage-How-tail-latency-impacts-customer-facing-applications#:~:text=Tail%20latency%20is%20the%20small,bulk%20of%20its%20response%20times.&text=That%201%25%20is%20the%20tail%20latency%20problem%20for%20the%20shepherd.>
- [3] <https://www.section.io/blog/preventing-long-tail-latency/>
- [4] Avinash Lakshman , Prashant Malik,Cassandra: a decentralized structured storage system, ACM SIGOPS Operating Systems Review, vol. 44, no.2, April 2010.
- [5] L. Suresh, M. Canini, S. Schmid, and A. Feldmann.,C3:Cutting tail latency in cloud data stores via adaptive replica selection, In Proceedings of the 12th USENIX Conference on Networked Systems Design and Implementation, 2015.
- [6] Jiang, W., Xie, H., Zhou, X., Fang, L. and Wang, J. Performance Analysis and Improvement of Replica Selection Algorithms for Key-Value Stores. 2017 IEEE 10th Int. Conf. on Cloud Computing (CLOUD), Honolulu, CA, USA,, 25-30 June, 2017.
- [7] Vikas Jaiman, Sonia Ben Mokhtar, Vivien Quéma, Lydia Chen, and Etienne Rivière. 2018. Héron: Taming Tail Latencies in Key-Value Stores under Heterogeneous Workloads. In International Symposium on Reliable Distributed Systems (SRDS) , 2018.
- [8] Larsen, S. and Lee, B.,Survey on System I/O Hardware Transactions and Impact on Latency, Throughput, andOther Factors., Advances in Computers, vol. 92, pp. 67-104.,doi:10.1016/B978-0-12-420232-0.00002-7, 2014.
- [9] <https://github.com/apache/cassandra>
- [10] <https://github.com/Santa-Maria-Shithil/prediction-based-replica-selection-strategy>
- [11] Brian F. Cooper , Adam Silberstein , Erwin Tam , Raghu Ramakrishnan , Russell Sears,Benchmarking cloud serving systems with YCSB, Proceedings of the 1st ACM symposiumon Cloud computing, June 10-11, 2010, Indianapolis, Indiana,USA.
- [12] S. M. Shithil and M. A. Adnan, "A Prediction Based Replica Selection Strategy for Reducing Tail Latency in Distributed Systems," 2020 IEEE 13th International Conference on Cloud Computing (CLOUD), Beijing, 2020, pp. 522-526, doi:10.1109/CLOUD49709.2020.00078

Thank you all

Questions?